

Introduction to LLMs in Python — Solutions

Reference answers for the exercises in `study-guide.pdf`. **Try each exercise first** — the learning is in the attempt. These are also in `llm-exercises/solutions/`, verified by `python check.py --solutions`.

2 · Using pre-trained models with pipelines

Exercise 1 · Summarize with a pipeline

Task. Return a ≤ 40 -word summary of `text` using a `summarization` pipeline.

Answer — `solutions/ex01_summarize.py`:

```
from transformers import pipeline

def solve(text):
    summarizer = pipeline(task="summarization", model="sshleifer/distilbart-cnn-12-6")
    out = summarizer(text, max_length=40)
    return out[0]["summary_text"]
```

Blanks filled — what and why:

- `task="summarization"` — the pipeline **task name**. `pipeline` looks this up in its task registry to know which kind of model and pre/post-processing to set up; `"summarization"` selects the summarization pipeline (a sequence-to-sequence job).
- `out[0]["summary_text"]` — the **output key**. A summarization pipeline returns a one-element list of dicts, e.g. `[{"summary_text": "..."}]`; `out[0]` is that dict and `["summary_text"]` reads the summary string out of it.

Exercise 2 · Generate text (control length + padding)

Task. Return ~ 30 **new** tokens of continuation from `distilgpt2`, with **no** warnings.

Answer — `solutions/ex02_generate.py`:

```

from transformers import pipeline

def solve(prompt):
    gen = pipeline(task="text-generation", model="distilgpt2")
    out = gen(prompt, max_new_tokens=30,
               pad_token_id=gen.tokenizer.eos_token_id, truncation=True)
    return out[0]["generated_text"]

```

Blanks filled — what and why:

- `max_new_tokens=30` — how many **new** tokens to generate (~30). We use `max_new_tokens`, **not** `max_length`, because the text-generation pipeline already sets a default `max_new_tokens`; also passing `max_length` makes the two conflict (a warning) and the length cap is ignored. `max_new_tokens` counts only the generated tokens, not the prompt.
- `pad_token_id=gen.tokenizer.eos_token_id` — the **padding token id** — worth unpacking:
 - **What padding is.** The model often processes several sequences at once (a *batch*), but they have different lengths. To line them up in one rectangular array, the short ones are filled up to the longest with a *filler* token — the **pad token**. The model ignores those filler positions via the *attention mask*, so `pad_token_id` just says *which id to use as filler*.
 - **GPT-2's problem.** GPT-2 was trained on continuous text with no dedicated pad token, so `tokenizer.pad_token_id` is `None`. When generation needs a pad id and finds none, it prints the “no padding token is set” warning.
 - **The fix.** GPT-2 *does* have an *end-of-sequence* token, `<|endoftext|>` (id 50256), exposed as `eos_token_id`. Setting `pad_token_id=gen.tokenizer.eos_token_id` reuses that id as the filler. It's safe because padding always comes **after** the real text (where “end of text” fits) and is masked out anyway — so it changes nothing in the output and simply silences the warning.
 - **Reading the expression:** `gen` = the pipeline, `gen.tokenizer` = its tokenizer, `.eos_token_id` = the integer 50256. So the line means “use 50256 as the pad id.”

Exercise 3 · Translate with a pipeline

Task. Return the English→Spanish translation of `text`.

Answer — `solutions/ex03_translate.py`:

```

from transformers import pipeline

def solve(text):
    translator = pipeline(task="translation_en_to_es",
                          model="Helsinki-NLP/opus-mt-en-es")
    return translator(text)[0]["translation_text"]

```

Blanks filled — what and why:

- `task="translation_en_to_es"` — the pipeline **task name**. Translation tasks are named `translation_<src>_to_<tgt>`, so English→Spanish is `translation_en_to_es`. It tells `pipeline` this is a translation job and in which direction; it agrees with the EN→ES model already given, and the result comes back under the `translation_text` key.

3 · The Transformer, in three shapes

Exercise 4 · Pick the right transformer family

Task. Map a task key (`"sentiment"`, `"poem"`, `"translation"`, `"extractive-qa"`) to `"encoder-only"` / `"decoder-only"` / `"encoder-decoder"` . *Offline.*

Answer — `solutions/ex04_family.py`:

```
def solve(task):
    families = {"sentiment": "encoder-only", "poem": "decoder-only",
               "translation": "encoder-decoder", "extractive-qa": "encoder-only"}
    return families[task]
```

Blanks filled — what and why:

- `"sentiment": "encoder-only"` — sentiment analysis reads the whole text and returns **one label** — pure *understanding*, nothing generated — so a bidirectional **encoder** (BERT-style) is the fit.
- `"poem": "decoder-only"` — writing a poem is open-ended *generation*, produced one token after another, which is exactly what an autoregressive **decoder** (GPT-style) does.
- `"translation": "encoder-decoder"` — translation is *sequence-to-sequence*: read the full source sentence, then write the target. The **encoder** understands the input and the **decoder** generates the output (attending back to the input via cross-attention) — e.g. Marian or T5.
- `"extractive-qa": "encoder-only"` — extractive QA picks the answer **span** out of a given context (it predicts the start and end positions), so it *understands* rather than generates — again a bidirectional **encoder** (e.g. BERT fine-tuned on SQuAD).

Exercise 5 · Classify sentiment

Task. Return `"POSITIVE"` / `"NEGATIVE"` for `text` using a fine-tuned sentiment pipeline.

Answer — `solutions/ex05_sentiment.py`:

```
from transformers import pipeline

def solve(text):
    clf = pipeline(task="sentiment-analysis",
                   model="distilbert-base-uncased-finetuned-sst-2-english")
    return clf(text)[0]["label"]
```

Blanks filled — what and why:

- `task="sentiment-analysis"` — the **task name** — an alias of text-classification specialised for sentiment; it tells `pipeline` to build a classification pipeline.
- `clf(text)[0]["label"]` — the pipeline returns `[{"label": ..., "score": ...}]; [0]` `["label"]` reads the predicted class (POSITIVE/NEGATIVE).

4 · From pipelines to Auto classes: datasets & tokenization

Exercise 6 · Load an Auto model + tokenizer

Task. Return `(tokenizer, model)`, where the model is an `AutoModelForSequenceClassification` with 2 labels.

Answer — `solutions/ex06_load_model.py`:

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

def solve():
    name = "distilbert-base-uncased"
    tokenizer = AutoTokenizer.from_pretrained(name)
    model = AutoModelForSequenceClassification.from_pretrained(name, num_labels=2)
    return tokenizer, model
```

Blanks filled — what and why:

- `_____ = AutoModelForSequenceClassification` — of the two imported classes, this is the one that adds a **classification head**; plain `AutoModel` would output only embeddings, with no way to predict a label.
- `num_labels=2` — two output classes (POSITIVE/NEGATIVE) — this sizes the classification head.

Exercise 7 · Tokenize texts (padding, truncation, tensors)

Task. Return padded, truncated (max 64) PyTorch tensors for a list of `texts`.

Answer — `solutions/ex07_tokenize.py`:

```
from transformers import AutoTokenizer

def solve(tokenizer, texts):
    return tokenizer(texts, return_tensors="pt", padding=True,
                    truncation=True, max_length=64)
```

Blanks filled — what and why:

- `return_tensors="pt"` — return **PyTorch** tensors ("pt"); "tf" / "np" would give TensorFlow/NumPy instead.
- `max_length=64` — cap each sequence at 64 tokens; combined with `truncation=True`, longer inputs are cut.

Exercise 8 • Tokenize a whole dataset with `.map`

Task. Tokenize a dataset row-by-row with `.map`, processing it in batches.

Answer — `solutions/ex08_tokenize_dataset.py`:

```
from transformers import AutoTokenizer
from datasets import load_dataset

def solve(tokenizer):
    data = load_dataset("imdb", split="train[:1%]")

    def tok(batch):
        return tokenizer(batch["text"], padding=True, truncation=True, max_length=128)

    return data.map(tok, batched=True)
```

Blanks filled — what and why:

- `batched=True` — hand many rows to `tok` at once instead of one at a time — far faster, and what a fast tokenizer is built for.

What the output shows:

Running `python solutions/ex08_tokenize_dataset.py` prints something like:

```
columns: ['text', 'label', 'input_ids', 'attention_mask'] (input_ids + attention_mask are new)
rows: 250
--- one tokenized example ---
label      0
text      I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that
input_ids  [101, 1045, 12524, 1045, 2572, 8025, 1011, 3756, ...] ... (length 128 )
attention_mask: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1] ...
decoded back [CLS] i rented i am curious - yellow from my video store because of all the
```

Line by line:

- **columns** — the dataset kept `text` and `label` and gained `input_ids` + `attention_mask` (what `.map` added).
- **rows: 250** — 1% of IMDB's 25,000-example train split.
- **label: 0** — IMDB uses `0` = negative, `1` = positive. **Heads-up:** all 250 rows here are `0`, because the train split is **ordered by label** (its first half is negative); `train[:1%]` therefore gives an all-negative subset — fine for demoing tokenization, but you'd have to **shuffle** before actually training on it.
- **text** — the raw review: the human-readable input.
- **input_ids ... (length 128)** — the review as vocabulary ids; `101` is `[CLS]` (the start marker). The length is **128** because that's the `max_length` cap: this review was long, so it was truncated/filled to 128 tokens.
- **attention_mask** — all `1`s here: each of those positions is a **real token** (no padding, because the review reached the 128 cap). A shorter review would show trailing `0`s.
- **decoded back** — the first 16 ids turned back into text: `[CLS] i rented i am curious - yellow ...`. Notice the special `[CLS]` marker, everything **lower-cased** (DistilBERT is *uncased*), and `CURIOUS-YELLOW` becoming `curious - yellow` (the tokenizer splits on punctuation/subwords). This round-trip confirms `input_ids` really encode the text.

5 · Fine-tuning through training

Exercise 9 · Set up TrainingArguments

Task. Return `TrainingArguments`: output `./out`, evaluate each epoch, 3 epochs, lr `2e-5`, train/eval batch 8, weight decay 0.01. *Offline.*

Answer — `solutions/ex09_training_args.py`:

```
from transformers import TrainingArguments

def solve():
    return TrainingArguments(
        output_dir="./out", eval_strategy="epoch", num_train_epochs=3,
        learning_rate=2e-5, per_device_train_batch_size=8,
        per_device_eval_batch_size=8, weight_decay=0.01)
```

Blanks filled — what and why:

- `eval_strategy="epoch"` — run evaluation at the end of **each epoch**.
- `num_train_epochs=3` — three full passes over the training data.
- `learning_rate=2e-5` — the step size — `2e-5` is a typical value for fine-tuning Transformers.

- `per_device_train_batch_size=8` — 8 training examples per step (per device).
- `weight_decay=0.01` — light regularisation that discourages large weights, curbing overfitting.

Exercise 10 · Wire up the Trainer

Task. Return a `Trainer` built from the pieces (don't call `.train()`).

Answer — `solutions/ex10_build_trainer.py`:

```
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          Trainer, TrainingArguments)
from datasets import Dataset

def solve(model, args, train_ds, eval_ds, tokenizer):
    return Trainer(model=model, args=args, train_dataset=train_ds,
                  eval_dataset=eval_ds, tokenizer=tokenizer)
```

Blanks filled — what and why:

- `model=model, args=args, train_dataset=train_ds, eval_dataset=eval_ds` — each `Trainer` parameter takes the same-named argument that `solve` received; the `Trainer` bundles the model, the config, the two datasets and the tokenizer, and its `.train()` runs the loop (the file's demo actually fires two steps).

Exercise 11 · Use the fine-tuned model (from logits)

Task. Return `"POSITIVE"` / `"NEGATIVE"` for `text`, computed **from raw logits**.

Answer — `solutions/ex11_predict.py`:

```
import pathlib
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

def solve(text):
    model_dir = pathlib.Path(__file__).resolve().parent.parent / "finetuned-demo"
    tok = AutoTokenizer.from_pretrained(model_dir)
    model = AutoModelForSequenceClassification.from_pretrained(model_dir)
    inp = tok(text, return_tensors="pt", padding=True, truncation=True)
    with torch.no_grad():
        logits = model(**inp).logits
    pred = int(torch.argmax(logits, dim=1))
    return model.config.id2label[pred].upper()
```

Blanks filled — what and why:

- `dim=1` — logits have shape `[batch, num_labels]`, so taking the `argmax` over **axis 1** picks the winning class for each example.
- `.upper()` — `id2label[pred]` gives the label name; `.upper()` normalises it to `POSITIVE/NEGATIVE` to match what the task expects.